Machine Learning

Complete Practical Exam Study Guide

B.Sc. (H) Computer Science VI Semester / B.A. Programme V Semester / Generic Elective VII Semester

Based on the uploaded Machine Learning guidelines: DSC17 / DSC-A5 / GE7c, effective Academic Year 2024-25

Purpose of this guide

This PDF is written for one-day practical exam preparation. It starts from zero, explains every syllabus topic, gives practical code for all listed experiments, and includes viva-ready short and detailed answers.

Use this guide in this order: first understand the common ML workflow, then revise Units 1-5, then practise the 12 practical programs, then use the final revision checklist.

Syllabus Coverage Map

The uploaded guideline contains five theory units and twelve practical prediction/model-building tasks. The table below maps each syllabus item to where it is covered in this PDF.

Syllabus unit	Topics covered	Exam focus
Unit 1: Introduction	Basic definitions and concepts, key elements, supervised learning, unsupervised learning, introduction to reinforcement learning, applications of ML	Definitions, types of learning, workflow, examples
Unit 2: Preprocessing	Feature scaling, feature selection methods, dimensionality reduction using Principal Component Analysis	Why preprocessing is needed, scaling methods, selection vs extraction, PCA steps
Unit 3: Regression	Simple linear regression, multiple linear regression, gradient descent, overfitting, regularization, regression evaluation metrics	Formulas, assumptions, loss, MSE, R^2, Ridge/Lasso
Unit 4: Classification	Decision trees, Naive Bayes, logistic regression, KNN, perceptron, multilayer perceptron, neural networks, SVM, classification metrics	Algorithm working, confusion matrix, accuracy, recall, specificity, F1, AUC
Unit 5: Clustering	Approaches for clustering, distance metrics, K-means clustering, hierarchical clustering	Unsupervised grouping, distance, elbow method, dendrogram
Practicals	12 listed models: NB, simple/multiple/polynomial regression, Ridge/Lasso, logistic regression, ANN, KNN, decision tree, SVM, K-means, hierarchical	Aim, theory, dataset, code, output, errors, viva

One-Day Practical Exam Strategy

A practical exam usually tests whether you can choose the correct algorithm, prepare data, train the model, evaluate it, and explain the result. Even if the exact dataset changes, the structure of your answer remains almost the same.

Time	What to do	Why it matters
First 60-90 minutes	Revise the ML workflow, train-test split, cross-validation, metrics, and all formulas.	These are common to almost every practical.
Next 2-3 hours	Run or write the 12 practical codes once. Focus on imports, dataset loading, model creation, fit, predict, metrics.	Most mistakes happen in code structure, not theory.
Next 1-2 hours	Practise viva answers for each algorithm: purpose, working, assumptions, advantages, disadvantages, use case.	Viva often checks conceptual understanding.
Last 30 minutes	Revise final formulas, common errors, and model selection rules.	Helps in last-minute recall.

Universal Machine Learning Practical Workflow

Almost every practical in your syllabus follows the same pipeline. Learn this once and reuse it everywhere.

Step	Action	Example in Python
1	Import libraries	pandas, numpy, sklearn, matplotlib
2	Load dataset	CSV file, UCI dataset, data.gov.in dataset, or sklearn built-in dataset

Step	Action	Example in Python
3	Understand X and y	X = input features; y = target/output
4	Preprocess	Handle missing values, scale features, encode categories, select features
5	Split data	train_test_split(X, y, test_size=0.2, random_state=42)
6	Create model	LinearRegression(), GaussianNB(), KNeighborsClassifier(), etc.
7	Train model	model.fit(X_train, y_train)
8	Predict	y_pred = model.predict(X_test)
9	Evaluate	MSE/R^2 for regression; confusion matrix, accuracy, recall, F1, AUC for classification
10	Conclude	State whether model performs well and why

Formula / Must know

Training data: data used to teach the model.
Test data: unseen data used to evaluate the model.
Model: mathematical function that maps input X to output y.
Prediction: model's estimated output for new input.
Evaluation metric: numerical score used to judge model performance.

Unit 1: Introduction to Machine Learning

1.1 Meaning of Machine Learning

Machine Learning (ML) is a branch of Artificial Intelligence where a computer system learns patterns from data and uses those patterns to make predictions or decisions without being explicitly programmed for every possible case. In normal programming, a programmer writes exact rules. In ML, the programmer gives data and an algorithm, and the system learns the rules automatically.

Traditional programming	Machine learning
Human writes rules explicitly.	Algorithm learns rules from examples.
Input + rules -> output.	Input + output examples -> learned model.
Good when rules are fixed and clear.	Good when rules are complex, hidden, or data-driven.
Example: calculator formula.	Example: spam email detection.

1.2 Basic Terms

Term	Meaning	Example
Dataset	Collection of records used for learning.	Iris flower measurements.
Instance / sample	One row of data.	One flower record.
Feature	Input variable used for prediction.	Petal length, age, income.
Target / label	Output variable to predict.	Species, price, disease yes/no.
Model	Learned mathematical relationship.	Regression line or trained classifier.
Training	Process of learning from data.	Calling model.fit().
Testing	Checking performance on unseen data.	Calling model.predict() and metrics.
Loss function	Measures prediction error during training.	MSE, log loss.
Generalization	Ability to work well on new data.	Good test accuracy, not just train accuracy.
Hyperparameter	Setting chosen before training.	k in KNN, C in SVM, tree depth.

1.3 Key Elements of Machine Learning

Every ML system contains a few important elements. In viva, do not only list them; explain how they connect.

Element	Explanation	Why important
Data	Raw examples from which the model learns.	Bad data produces bad model.
Features	Useful measurable inputs extracted from data.	Better features improve prediction.
Algorithm	Learning method such as regression, tree, SVM, K-means.	Different problems need different algorithms.
Model	Final learned function after training.	Used for prediction.
Objective / loss	Numerical goal to minimize or maximize.	Guides learning.
Evaluation	Testing using metrics on unseen data.	Tells whether model is useful.
Deployment	Using the trained model in real application.	Turns experiment into product.

1.4 Types of Machine Learning

Type	What data is given?	Goal	Examples
Supervised learning	Inputs X and correct outputs y are given.	Predict output for new data.	Regression, classification.
Unsupervised learning	Only inputs X are given, no labels.	Find hidden structure or groups.	Clustering, PCA.
Reinforcement learning	Agent interacts with environment and receives rewards.	Learn best actions to maximize reward.	Game AI, robotics, self-driving simulation.

Supervised Learning

Supervised learning is used when the dataset contains the correct answer for each training example. If the output is numerical, the task is regression. If the output is a category, the task is classification.

Unsupervised Learning

Unsupervised learning is used when the dataset has no target label. The algorithm tries to discover patterns, groups, or compressed representations. K-means and hierarchical clustering are unsupervised algorithms. PCA is also usually treated as an unsupervised dimensionality reduction technique.

Reinforcement Learning

Reinforcement learning is different from ordinary supervised learning. There may not be a fixed correct label for every action. Instead, an agent performs actions in an environment and receives rewards or penalties. Over time, it learns a policy: a rule for choosing actions.

1.5 Applications of Machine Learning

Domain	ML application	Typical model type
Healthcare	Disease prediction, medical image classification	Classification, neural networks
Finance	Credit risk, fraud detection, stock trend analysis	Classification, regression, anomaly detection
E-commerce	Recommendation systems, demand forecasting	Clustering, regression, classification
Education	Student performance prediction	Regression/classification
Agriculture	Crop yield prediction, disease detection	Regression/classification
Transport	Traffic prediction, route optimization	Regression, reinforcement learning
Natural language	Spam detection, sentiment analysis	Naive Bayes, SVM, neural networks
Image processing	Face recognition, object detection	Neural networks

1.6 Ten Examples for Unit 1

No.	Situation	ML type	Step-by-step reasoning
1	Predict house price from area	Supervised regression	Input area is X, price is y, output is numerical, so use regression.
2	Classify email as spam/not spam	Supervised classification	Training emails have labels; output is category.
3	Group customers by buying behavior	Unsupervised clustering	No correct group labels are given; algorithm discovers groups.
4	Reduce 100 exam features to 2 important components	Unsupervised dimensionality reduction	PCA compresses features while preserving variance.
5	Robot learns to move without hitting walls	Reinforcement learning	Robot gets reward for safe movement and penalty for collision.
6	Predict marks from study hours	Supervised regression	Marks are numerical target values.
7	Predict whether a tumor is benign/malignant	Supervised classification	Target has discrete classes.
8	Detect unusual bank transactions	Often unsupervised/anomaly detection	Fraud labels may be missing, so unusual patterns are detected.
9	Recommend movies	Supervised/unsupervised hybrid	Uses past user behavior to predict preference.
10	Game-playing AI improves through trial and error	Reinforcement learning	Agent learns actions that maximize score.

Formula / Must know

Regression = predict continuous numerical value.
Classification = predict category/class label.
Clustering = group similar data without labels.
Reinforcement learning = learn action policy from reward feedback.

How to Answer in Exam

Short answer	Machine Learning is a technique where computers learn patterns from data and make predictions without explicit rule-based programming.
Detailed answer	Define ML, contrast it with traditional programming, mention dataset, features, target, model, training, testing, then give one regression and one classification example.
Common mistake	Do not say ML means only AI or only robots. ML is data-driven pattern learning.

Unit 1 Viva Questions

Viva question	Correct answer / framing
What is Machine Learning?	It is a method where algorithms learn patterns from data and use them to make predictions or decisions on new data.
What is a feature?	A feature is an input variable used by the model, such as age, area, petal length, or income.
What is the difference between training and testing?	Training learns model parameters; testing evaluates performance on unseen data.
What is supervised learning?	Learning from labelled data where both input and correct output are available.
What is unsupervised learning?	Learning hidden structure from unlabelled data, such as clustering similar records.
What is reinforcement learning?	An agent learns actions by interacting with an environment and receiving rewards or penalties.
Regression vs classification?	Regression predicts continuous values; classification predicts categories.
Why split data?	To test generalization on unseen data and avoid judging the model only on memorized training data.
What is generalization?	The ability of a model to perform well on new, unseen data.
Give two ML applications.	Spam detection and house price prediction; many other examples are valid if type and target are clear.

Unit 2: Preprocessing

Preprocessing means preparing raw data before giving it to a machine learning algorithm. Real data may contain different scales, missing values, irrelevant features, duplicate features, noise, or too many dimensions. Preprocessing improves model accuracy, speed, and reliability.

2.1 Why Preprocessing is Needed

- Algorithms such as KNN, SVM, gradient descent, PCA, and neural networks are sensitive to feature scale.
- Irrelevant features can confuse the model and increase overfitting.
- High-dimensional data increases computation and may reduce performance.
- Preprocessing makes data cleaner, comparable, and easier for algorithms to learn from.

2.2 Feature Scaling

Feature scaling changes numeric columns to a comparable scale. Suppose one feature is age from 0 to 100 and another is salary from 10,000 to 1,00,000. Distance-based algorithms may give too much importance to salary because its numeric range is bigger. Scaling prevents this unfair dominance.

Method	Formula / idea	When used	Example
Standardization	z = (x - mean) / standard deviation	When data may have outliers or algorithms assume centered data.	SVM, Logistic Regression, PCA, Neural Networks
Min-Max Scaling	x_scaled = (x - min) / (max - min)	When values should be between 0 and 1.	KNN, neural networks
Robust Scaling	Uses median and interquartile range.	When outliers are present.	Income data with extreme values
Normalization	Scales vector length, often to unit norm.	Text mining or cosine similarity tasks.	Document classification

Formula / Must know

Standardization: z = (x - mean) / standard_deviation
Min-Max scaling: x' = (x - min) / (max - min)
Important: Fit scaler on training data only, then transform train and test.

2.3 Feature Selection Methods

Feature selection means choosing the most useful input columns and removing irrelevant or redundant ones. It does not create new features; it selects from existing features. This can reduce overfitting, improve speed, and make the model easier to explain.

Method type	How it works	Examples	Advantage	Disadvantage
Filter methods	Select features using statistical scores before training the model.	Correlation, chi-square, ANOVA, mutual information	Fast and simple	May ignore model-specific interactions
Wrapper methods	Train model repeatedly with different feature subsets.	Forward selection, backward elimination, RFE	Often accurate	Computationally expensive

Method type	How it works	Examples	Advantage	Disadvantage
Embedded methods	Feature selection happens during model training.	Lasso, tree feature importance	Model-aware	Depends on chosen algorithm

2.4 Dimensionality Reduction: Principal Component Analysis (PCA)

Dimensionality reduction means reducing the number of input features while keeping as much useful information as possible. PCA is a popular linear dimensionality reduction technique. It transforms original correlated features into new uncorrelated features called principal components.

Working of PCA

Step	Explanation
1. Standardize features	PCA is scale-sensitive, so features must be centered and scaled.
2. Find variance directions	PCA finds directions in which data varies the most.
3. Create principal components	First principal component captures maximum variance, second captures next maximum variance and is perpendicular to first.
4. Choose number of components	Keep enough components to preserve desired variance, e.g., 95%.
5. Transform data	Original features are converted into fewer principal component features.

Feature Selection vs PCA

Point	Feature selection	PCA
Meaning	Chooses original features.	Creates new transformed features.
Interpretability	More interpretable because features stay original.	Less interpretable because components are combinations.
Use	When you want to know important columns.	When you want compact representation and speed.
Example	Choose age and income only.	Create PC1 and PC2 from many numeric features.

2.5 Ten Examples for Unit 2

No.	Example	What to apply	Reason
1	Age ranges 0-80 and income ranges 20,000-2,00,000	StandardScaler/MinMaxScaler	Prevents income from dominating distance.
2	KNN on height and weight	Feature scaling	KNN uses distance, so scale matters.
3	SVM for tumor classification	Standardization	SVM margin depends on feature scale.
4	100 features but only 10 useful	Feature selection	Removes noise and reduces overfitting.
5	Highly correlated columns: total marks and percentage	Remove one correlated feature	Redundant features add little information.
6	Dataset has 500 image pixel features	PCA	Compresses high-dimensional data.
7	Need explainable model for medical exam	Feature selection	Original selected features are easier to explain.
8	Need 2D visualization of 4-feature Iris dataset	PCA to 2 components	Allows plotting in 2D.
9	Outliers in income column	RobustScaler	Median/IQR are less affected by outliers.
10	Neural network trains slowly	Feature scaling	Gradient descent converges faster with scaled features.

Formula / Must know

Preprocessing rule: split first, then fit preprocessing on training data only.
Feature selection keeps original variables; PCA creates new variables.
Scaling is essential for distance-based and gradient-based methods.

How to Answer in Exam

Short answer	Preprocessing prepares raw data by scaling, selecting features, and reducing dimensions before model training.
Detailed answer	Explain why raw data can be unsuitable, then describe scaling, feature selection, and PCA with one example each. Mention that preprocessing should be fitted on training data only.
Common mistake	Do not scale the full dataset before train-test split in an exam answer; it causes data leakage.

Unit 2 Viva Questions

Viva question	Correct answer / framing
What is preprocessing?	Preparing raw data so ML algorithms can learn effectively.
Why is scaling required?	To bring features to comparable numeric ranges, especially for distance and gradient-based algorithms.
Name two scaling methods.	Standardization and Min-Max scaling.
Which algorithms need scaling?	KNN, SVM, PCA, logistic regression with gradient descent, neural networks.
What is feature selection?	Selecting useful input columns and removing irrelevant or redundant ones.
What are filter methods?	Feature selection based on statistical tests such as correlation, chi-square, ANOVA.
What is PCA?	A dimensionality reduction method that transforms features into principal components capturing maximum variance.
Does PCA keep original features?	No. It creates new components from combinations of original features.
Why standardize before PCA?	Because PCA depends on variance and large-scale features can dominate components.
What is data leakage?	When information from test data influences training, producing over-optimistic results.

Unit 3: Regression

Regression is supervised learning used when the target/output is a continuous numerical value. Examples include predicting marks, salary, price, temperature, or sales. The model learns a mathematical relationship between input features X and numerical target y.

3.1 Simple Linear Regression

Simple linear regression predicts a numerical output using one input feature. It fits a straight line through the data. The goal is to choose the best slope and intercept so that predicted values are close to actual values.

> **Formula / Must know**
>
> Simple linear regression: y_hat = b0 + b1*x
> b0 = intercept, b1 = slope/coefficient
> Error/residual = y - y_hat

Working

- Take one feature x and target y.
- Assume the relationship is approximately linear.
- Find the line that minimizes total squared error.
- Use the learned line to predict y for new x values.

3.2 Multiple Linear Regression

Multiple linear regression predicts a numerical output using more than one input feature. It is an extension of simple linear regression. The model learns one coefficient for each feature.

> **Formula / Must know**
>
> Multiple linear regression: y_hat = b0 + b1*x1 + b2*x2 + ... + bp*xp
> Each coefficient tells the average effect of that feature when other features are fixed.

3.3 Polynomial Regression

Polynomial regression is used when the relationship between x and y is curved rather than straight. It creates polynomial features such as x^2, x^3, and then applies linear regression on those transformed features.

> **Formula / Must know**
>
> Polynomial model example: y_hat = b0 + b1*x + b2*x^2 + b3*x^3

3.4 Gradient Descent

Gradient descent is an optimization algorithm used to minimize a loss function. In regression, it can be used to find coefficients that minimize error. Imagine standing on a hill and taking small steps downhill until reaching a low point. The step size is controlled by the learning rate.

Term	Meaning
Loss function	Measures how wrong the model is.
Gradient	Direction and amount by which parameters should change.
Learning rate	Size of each update step.
Iteration/epoch	One update cycle over data.
Convergence	When loss stops decreasing significantly.

Formula / Must know

Parameter update idea: new_weight = old_weight - learning_rate * gradient
If learning rate is too high: may overshoot and diverge.
If learning rate is too low: training becomes very slow.

3.5 Overfitting and Underfitting

Concept	Meaning	Symptoms	Solution
Underfitting	Model is too simple to capture pattern.	Poor training and poor test performance.	Use better features, more complex model, reduce regularization.
Overfitting	Model memorizes training data/noise.	Very high train score but low test score.	Regularization, cross-validation, simpler model, more data.
Good fit	Model captures true pattern and generalizes.	Good train and test performance.	Balanced complexity.

3.6 Regularization: Ridge and Lasso

Regularization adds a penalty to the loss function to discourage very large coefficients. It helps reduce overfitting. Ridge uses L2 penalty; Lasso uses L1 penalty and can make some coefficients exactly zero, so it can also perform feature selection.

Method	Penalty	Effect	When useful
Ridge Regression	L2 penalty: sum of squared coefficients	Shrinks coefficients but usually does not make them zero.	When many features contribute.
Lasso Regression	L1 penalty: sum of absolute coefficients	Can shrink some coefficients to exactly zero.	When feature selection is useful.

3.7 Regression Evaluation Metrics

A regression model must be evaluated using numerical error metrics. Your syllabus specifically mentions MSE and R^2 score.

Formula / Must know

Mean Squared Error (MSE) = (1/n) * sum(yi - y_hat_i)^2
R^2 score = 1 - SS_res / SS_total
Lower MSE is better. Higher R^2 is better. R^2 close to 1 means strong fit.

Metric	Meaning	Good value	Weakness
MSE	Average squared prediction error.	Lower is better.	Sensitive to outliers because errors are squared.

Metric	Meaning	Good value	Weakness
RMSE	Square root of MSE.	Lower is better.	Still sensitive to outliers.
MAE	Average absolute error.	Lower is better.	Does not strongly penalize large errors.
R^2	Proportion of variance explained by model.	Closer to 1 is better.	Can mislead if overfitting or wrong model assumptions.

3.8 Ten Regression Examples

No.	Problem	Model choice	Reasoning
1	Predict marks from study hours	Simple linear regression	One numeric input, numeric output.
2	Predict house price from area, rooms, location score	Multiple regression	Multiple inputs, numeric target.
3	Predict salary from experience	Simple regression	Experience often has linear trend with salary.
4	Predict crop yield from rainfall and fertilizer	Multiple regression	Several factors affect yield.
5	Curved relation between speed and fuel consumption	Polynomial regression	Nonlinear curve can be modelled by polynomial terms.
6	Too many correlated features in price prediction	Ridge regression	L2 penalty stabilizes coefficients.
7	Need automatic feature removal	Lasso regression	L1 penalty can set coefficients to zero.
8	Train error low but test error high	Regularization/cross-validation	This indicates overfitting.
9	MSE extremely large due to outliers	Check outliers/MAE	Squared error penalizes large errors heavily.
10	R^2 is 0.85	Good regression fit	Model explains about 85% of target variance.

How to Answer in Exam

Short answer	Regression predicts a continuous numerical target. Linear regression fits a line/hyperplane by minimizing error.
Detailed answer	Mention X, y, model equation, training objective, MSE/R^2 evaluation, and one example such as house price prediction. Also explain overfitting and regularization when asked.
Common mistake	Do not confuse regression with logistic regression; logistic regression is used for classification despite its name.

Unit 3 Viva Questions

Viva question	Correct answer / framing
What is regression?	Supervised learning used to predict continuous numeric values.
Equation of simple linear regression?	y_hat = b0 + b1*x.
What is multiple linear regression?	Linear regression with two or more input features.
What is gradient descent?	An optimization method that updates parameters in the direction that reduces loss.
What is learning rate?	Step size used during gradient descent updates.
What is overfitting?	Model performs very well on training data but poorly on unseen data because it memorized noise.
How to reduce overfitting?	Regularization, cross-validation, simpler model, more data, feature selection.
Difference between Ridge and Lasso?	Ridge uses L2 penalty; Lasso uses L1 penalty and can make coefficients zero.
What is MSE?	Average squared difference between actual and predicted values.
What is R^2 score?	Proportion of variance in target explained by the model; closer to 1 is better.

Unit 4: Classification

Classification is supervised learning used when the target is a class/category. Examples: spam/not spam, disease/no disease, pass/fail, iris species, fraud/not fraud. A classifier learns decision boundaries that separate classes.

4.1 Classification Evaluation Metrics

Before learning algorithms, understand evaluation. In classification, a prediction can be correct or wrong in different ways. The confusion matrix is the base of most classification metrics.

Actual / Predicted	Predicted Positive	Predicted Negative
Actual Positive	TP: correctly predicted positive	FN: positive case missed
Actual Negative	FP: negative case wrongly predicted positive	TN: correctly predicted negative

Formula / Must know

Accuracy = (TP + TN) / (TP + TN + FP + FN)
Error rate = (FP + FN) / total = 1 - Accuracy
Recall/Sensitivity = TP / (TP + FN)
Specificity = TN / (TN + FP)
Precision = TP / (TP + FP)
F1-score = 2 * Precision * Recall / (Precision + Recall)
AUC = area under ROC curve; higher AUC means better class separation.

4.2 Decision Trees

A decision tree is a flowchart-like model that splits data using feature conditions. Internal nodes are questions, branches are answers, and leaf nodes are final class predictions. It is easy to explain and visualize.

Concept	Meaning
Root node	First split of the tree.
Internal node	A decision condition on a feature.
Leaf node	Final predicted class.
Gini/entropy	Impurity measures used to choose good splits.
Pruning	Reducing tree complexity to avoid overfitting.

4.3 Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on Bayes theorem. It is called naive because it assumes features are conditionally independent given the class. This assumption is often not fully true, but the algorithm still works well in many text classification tasks.

Formula / Must know

Bayes theorem: P(Class | Features) = P(Features | Class) * P(Class) / P(Features)
Choose the class with the highest posterior probability.

4.4 Logistic Regression

Logistic regression is a classification algorithm, not a regression algorithm for continuous values. It predicts probability using the sigmoid function. For binary classification, if probability is >= 0.5, class is usually predicted as 1; otherwise 0.

Formula / Must know

Sigmoid function: p = 1 / (1 + e^(-z))
z = b0 + b1*x1 + ... + bp*xp
Decision rule: if p >= threshold, predict positive class.

4.5 K-Nearest Neighbor Classifier

KNN is a simple distance-based algorithm. To classify a new point, it finds the k closest training points and assigns the majority class among those neighbors. KNN does not learn explicit parameters; it stores training data and compares at prediction time.

Point	Explanation
k	Number of neighbors considered.
Distance	Usually Euclidean distance for numeric features.
Scaling	Very important because KNN is distance-based.
Small k	Can be noisy and overfit.
Large k	Smoother but may underfit.

4.6 Perceptron

A perceptron is one of the simplest neural network models. It is a linear binary classifier. It calculates a weighted sum of inputs and applies an activation/step function to decide the class. It can solve linearly separable problems but fails on non-linearly separable problems such as XOR.

Formula / Must know

Perceptron idea: output = activation(w1*x1 + w2*x2 + ... + b)

4.7 Multilayer Perceptron and Neural Networks

A multilayer perceptron (MLP) is a neural network with an input layer, one or more hidden layers, and an output layer. Hidden layers allow the network to learn nonlinear relationships. Training usually uses forward propagation to compute predictions and backpropagation with gradient descent to update weights.

Term	Meaning
Neuron	Small computation unit that applies weights, bias, and activation.
Layer	Group of neurons.
Activation function	Adds nonlinearity, e.g., ReLU, sigmoid.
Forward propagation	Computing output from input through layers.
Backpropagation	Computing error gradients and updating weights.
Epoch	One full pass through training data.

4.8 Support Vector Machine (SVM)

SVM is a powerful classification algorithm that tries to find the best separating boundary between classes. This boundary is called a hyperplane. SVM chooses the hyperplane with maximum margin, meaning it tries to keep classes as far apart as possible from the boundary.

Concept	Meaning
Hyperplane	Decision boundary separating classes.
Margin	Distance between boundary and nearest points.
Support vectors	Training points closest to the boundary; they influence the hyperplane.
Kernel	Function that allows SVM to work in higher-dimensional feature space.
C parameter	Controls trade-off between margin and classification errors.

4.9 Algorithm Comparison

Algorithm	Strength	Weakness	Scaling needed?
Decision Tree	Easy to explain; handles nonlinear rules.	Can overfit if deep.	Usually no
Naive Bayes	Fast; good for text.	Independence assumption may be unrealistic.	Usually no
Logistic Regression	Simple probabilities; interpretable.	Linear decision boundary unless features transformed.	Recommended
KNN	Simple; no training phase.	Slow prediction; sensitive to scale and k.	Yes
Perceptron	Simple linear classifier.	Cannot solve nonlinear cases alone.	Recommended
MLP/ANN	Learns nonlinear patterns.	Needs tuning, data, and scaling.	Yes
SVM	Strong margin-based classifier.	Can be slow on large datasets.	Yes

4.10 Ten Classification Examples

No.	Problem	Suitable algorithm	Reasoning
1	Spam email detection	Naive Bayes/SVM	Text classification often works well with probabilistic or margin classifiers.
2	Iris species prediction	KNN/Decision Tree/SVM	Small multiclass dataset with numeric features.
3	Tumor benign/malignant	Logistic Regression/SVM	Binary classification with important recall.
4	Loan approval	Decision Tree/Logistic Regression	Explainability is useful.
5	Handwritten digit classification	ANN/MLP	Nonlinear patterns in images.
6	Pass/fail prediction	Logistic Regression	Binary outcome with probability interpretation.
7	Customer churn	Decision Tree/Logistic Regression	Predict whether customer will leave.
8	Fraud detection	SVM/Tree models	Needs separation of rare suspicious cases.
9	Classify fruits by weight/color	KNN	Distance between feature values can help.
10	Medical screening	Classifier with high recall	Missing true disease cases is costly.

How to Answer in Exam

Short answer	Classification predicts discrete class labels. It is evaluated using confusion matrix-based metrics such as accuracy, recall, specificity, F1-score, and AUC.
Detailed answer	Define classification, give an example, explain TP/TN/FP/FN, then describe the requested algorithm's working, advantages, disadvantages, and metric choice.
Common mistake	Do not rely only on accuracy for imbalanced datasets; explain recall, precision, F1, or AUC when needed.

Unit 4 Viva Questions

Viva question	Correct answer / framing
What is classification?	Supervised learning for predicting categories/classes.
What is a confusion matrix?	A table comparing actual and predicted classes using TP, TN, FP, FN.
What is recall?	TP / (TP + FN); ability to detect actual positives.
What is specificity?	TN / (TN + FP); ability to detect actual negatives.
Why F1-score?	It balances precision and recall, useful when classes are imbalanced.
What is Naive Bayes based on?	Bayes theorem with conditional independence assumption.
Why is logistic regression used for classification?	It outputs probabilities using sigmoid function and applies a threshold.
Why scale data for KNN and SVM?	They depend on distances/margins, so feature scale affects results.
What are support vectors?	Points closest to the SVM hyperplane that determine the margin.
What is backpropagation?	A neural network training method that computes gradients of error and updates weights.

Unit 5: Clustering

Clustering is an unsupervised learning technique used to group similar data points. Unlike classification, clustering does not use predefined labels. The algorithm discovers groups based on similarity or distance.

5.1 Approaches for Clustering

Approach	Meaning	Example algorithm
Partitioning	Divides data into k groups.	K-means
Hierarchical	Builds a tree of clusters.	Agglomerative clustering
Density-based	Finds dense regions separated by sparse regions.	DBSCAN
Model-based	Assumes data is generated by statistical distributions.	Gaussian Mixture Models

5.2 Distance Metrics

Clustering needs a way to measure similarity. For numeric data, similarity is usually calculated using a distance metric. Smaller distance means points are more similar.

Metric	Formula idea	Use
Euclidean	Straight-line distance	Most common for K-means and numeric data.
Manhattan	Sum of absolute coordinate differences	Grid-like movement or high-dimensional sparse data.
Minkowski	Generalized form of Euclidean/Manhattan	Flexible distance metric.
Cosine distance	Based on angle between vectors	Text/document similarity.

5.3 K-means Clustering

K-means partitions data into k clusters. The user chooses k in advance. The algorithm repeatedly assigns each point to the nearest centroid and then updates centroids until assignments become stable.

Working of K-means

Step	Action
1	Choose number of clusters k.
2	Initialize k centroids.
3	Assign each data point to nearest centroid.
4	Recalculate centroid as mean of assigned points.
5	Repeat assignment and update until convergence.

Formula / Must know

K-means objective: minimize within-cluster sum of squared distances.
Elbow method: plot inertia/WCSS for different k and choose the elbow point.
K-means is sensitive to feature scaling and initial centroids.

5.4 Hierarchical Clustering

Hierarchical clustering creates a hierarchy/tree of clusters. Agglomerative hierarchical clustering starts with each point as a separate cluster and repeatedly merges the closest clusters. The result can be shown using a dendrogram.

Linkage	Meaning
Single linkage	Distance between closest points of two clusters.
Complete linkage	Distance between farthest points of two clusters.
Average linkage	Average distance between points in two clusters.
Ward linkage	Merges clusters to minimize increase in variance.

5.5 K-means vs Hierarchical

Point	K-means	Hierarchical
Need k before training?	Yes	Not always; dendrogram helps decide.
Output	Flat clusters	Tree/dendrogram plus clusters.
Speed	Fast for larger datasets	Slower for large datasets.
Shape assumption	Works best for spherical clusters	More flexible depending on linkage.
Interpretability	Centroids explain clusters	Dendrogram explains merging structure.

5.6 Ten Clustering Examples

No.	Problem	Clustering idea	Reasoning
1	Group customers by purchase behavior	K-means	Find segments for marketing.
2	Group documents by topic	Cosine distance clustering	Text vectors compared by angle.
3	Cluster iris flowers without species labels	K-means/hierarchical	Groups based on measurements.
4	Find student learning groups	K-means	Based on marks, attendance, activity.
5	Visualize cluster hierarchy	Hierarchical clustering	Dendrogram shows merging sequence.
6	Need quick segmentation of large data	K-means	Computationally efficient.
7	Unknown number of groups	Hierarchical + dendrogram	Can inspect tree to choose clusters.
8	Outlier points disturb clusters	Check scaling/outliers	Distances are affected by extreme values.
9	Choose best k	Elbow method	Look for point where inertia decrease slows.
10	Cluster countries by development indicators	K-means/PCA + clustering	Groups based on numeric indicators.

How to Answer in Exam

Short answer	Clustering is unsupervised grouping of similar data points without predefined labels.
Detailed answer	Explain that clustering uses similarity/distance, mention K-means or hierarchical working, give an example, and discuss how clusters are evaluated or interpreted.
Common mistake	Do not say clustering predicts a known label. It discovers groups; labels are not provided during training.

Unit 5 Viva Questions

Viva question	Correct answer / framing
What is clustering?	Unsupervised learning method that groups similar data points without labels.
What is a distance metric?	A method to measure how far or dissimilar two data points are.
How does K-means work?	Choose k, initialize centroids, assign points, update centroids, repeat until stable.
What is centroid?	Mean position of points in a cluster.
What is the elbow method?	A method to choose k by plotting inertia and selecting the point where improvement slows.
What is hierarchical clustering?	A clustering method that builds a tree-like hierarchy of clusters.
What is a dendrogram?	A tree diagram showing how clusters merge or split.
What is linkage?	Rule used to measure distance between clusters in hierarchical clustering.
Why scale before clustering?	Distance calculations are affected by feature scale.
K-means vs classification?	K-means has no labels and discovers groups; classification learns from known labels.

Practical Lab Preparation

Your guideline says students may use MATLAB/Octave/Python/R and public datasets. This guide uses Python with scikit-learn because it is common in college practicals, easy to write, and supports all listed algorithms. The same logic applies to other tools.

Common Libraries

Library	Use
numpy	Numerical arrays and calculations.
pandas	Tabular data handling if using CSV files.
matplotlib	Plots for regression line, clusters, elbow method, dendrogram.
scikit-learn / sklearn	Datasets, preprocessing, model training, metrics, cross-validation.
scipy	Dendrogram plotting for hierarchical clustering.

Common Code Skeleton

```
# 1. Import libraries
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score

# 2. Load dataset and create X, y
# X = input features, y = target label/value

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scale if algorithm needs it
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Create and train model
model.fit(X_train_scaled, y_train)

# 6. Predict and evaluate
y_pred = model.predict(X_test_scaled)
print("Evaluation score:", accuracy_score(y_test, y_pred))
```

Formula / Must know

For regression practicals: report MSE and R^2 score.
For classification practicals: report confusion matrix, accuracy, error rate, recall, specificity, F1-score, and AUC when possible.
For clustering practicals: show cluster labels, plot clusters when possible, and use elbow/dendrogram.

Practical 1: Naive Bayes Classifier

Aim

To build a Naive Bayes classification model and evaluate it using train-test split, confusion matrix, accuracy, recall, F1-score, and cross-validation.

Theory

Naive Bayes is a probabilistic classifier based on Bayes theorem. It calculates the probability of each class for a given input and selects the class with the highest posterior probability. GaussianNB is used when features are continuous numeric values and are assumed to follow a Gaussian distribution.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Iris dataset from sklearn. Features are sepal length, sepal width, petal length, petal width. Target is flower species.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score, precision_score,
    f1_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset into training and test data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create and train Naive Bayes model
model = GaussianNB()
model.fit(X_train, y_train)

# Predict classes for test data
y_pred = model.predict(X_test)

# Evaluate model
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred, average="weighted"))
print("Precision:", precision_score(y_test, y_pred, average="weighted"))
print("F1-score:", f1_score(y_test, y_pred, average="weighted"))

# k-fold cross-validation
scores = cross_val_score(model, X, y, cv=5)
print("5-fold CV scores:", scores)
print("Mean CV accuracy:", scores.mean())
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output will show a 3x3 confusion matrix because Iris has 3 classes, followed by accuracy, recall, precision, F1-score, and cross-validation scores. Accuracy is usually high on Iris.

Result / Conclusion Format

The Naive Bayes Classifier model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
IndentationError near y	Make sure y = iris.target starts at the beginning of the line.
Wrong average in recall_score	For multiclass, use average='weighted' or 'macro'.
Forgetting stratify	stratify=y keeps class distribution balanced in train/test split.

Practical Viva

Viva question	Correct answer / framing
Why is it called Naive?	Because it assumes features are conditionally independent given the class.
Which Naive Bayes variant is used here?	GaussianNB, suitable for continuous numeric features.
How do you frame result?	The model was trained on labelled Iris data and evaluated using accuracy and confusion matrix; high accuracy means most flowers were classified correctly.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 2: Simple Linear Regression

Aim

To predict a continuous value using one input feature and evaluate the model using MSE and R^2 score.

Theory

Simple linear regression fits a straight line y_hat = b0 + b1*x. It is used when one input feature is used to predict a continuous target. The model learns slope and intercept by minimizing squared error.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Diabetes dataset from sklearn. We use one feature to predict disease progression score.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load diabetes dataset
diabetes = load_diabetes()
X = diabetes.data[:, [2]]   # using one feature: BMI-like feature
y = diabetes.target

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train simple linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)

# Evaluate
print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_)
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

# Cross-validation using R2 score
cv_scores = cross_val_score(model, X, y, cv=5, scoring="r2")
print("5-fold R2 scores:", cv_scores)
print("Mean CV R2:", cv_scores.mean())

# Plot actual points and regression line
plt.scatter(X_test, y_test, label="Actual")
plt.plot(X_test, y_pred, label="Predicted line")
plt.xlabel("BMI feature")
plt.ylabel("Disease progression")
plt.legend()
plt.show()
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows intercept, coefficient, MSE, R^2 score, cross-validation R^2 values, and a scatter plot with regression line.

Result / Conclusion Format

The Simple Linear Regression model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
Plot line looks broken	Sort X_test before plotting if you want a clean line. Prediction is still correct.
R^2 not very high	One feature may not explain the target fully. Explain this in conclusion.
Using 1D X	scikit-learn expects X as 2D, so use diabetes.data[:, [2]] not diabetes.data[:, 2].

Practical Viva

Viva question	Correct answer / framing
Why use double brackets for X?	To keep X two-dimensional as required by sklearn estimators.
What does coefficient mean?	It shows change in predicted target for one-unit change in feature.
How to frame result?	The model predicts a numeric target from one feature; MSE measures error and R^2 shows explained variance.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 3: Multiple Linear Regression

Aim

To build a regression model using multiple input features and evaluate it using MSE, R^2, and cross-validation.

Theory

Multiple linear regression extends simple regression by using several features. It learns one coefficient for each feature and predicts a continuous target.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Diabetes dataset from sklearn. All numeric features are used to predict disease progression.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load data
diabetes = load_diabetes()
X = diabetes.data       # all features
y = diabetes.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

# Cross-validation
scores = cross_val_score(model, X, y, cv=5, scoring="r2")
print("5-fold R2 scores:", scores)
print("Mean CV R2:", scores.mean())
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows coefficients for all features, MSE, R^2, and 5-fold cross-validation R^2 scores.

Result / Conclusion Format

The Multiple Linear Regression model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
Too many coefficients	There is one coefficient for each feature; print feature names using diabetes.feature_names.
Negative R^2 in some folds	Model may perform worse than mean prediction in that fold.
Overfitting concern	Compare train score and test score; use Ridge/Lasso if needed.

Practical Viva

Viva question	Correct answer / framing
Simple vs multiple regression?	Simple uses one feature; multiple uses two or more features.
What is R^2?	It measures how much variance in target is explained by the model.
How to frame result?	The model uses multiple medical measurements to predict disease progression and is judged using MSE/R^2.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 4: Polynomial Regression

Aim

To model a nonlinear relationship by creating polynomial features and applying linear regression.

Theory

Polynomial regression transforms x into powers such as x^2 and x^3. Linear regression is then applied on these transformed features. It can fit curved patterns but high degree may overfit.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Synthetic curved dataset created using numpy so the polynomial relationship is clear.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

# Create a curved dataset
np.random.seed(42)
X = np.linspace(-3, 3, 100).reshape(-1, 1)
y = 2 * X.ravel()**2 + 3 * X.ravel() + 5 + np.random.randn(100) * 2

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create polynomial regression pipeline
model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

# Cross-validation
scores = cross_val_score(model, X, y, cv=5, scoring="r2")
print("5-fold R2 scores:", scores)
print("Mean CV R2:", scores.mean())

# Plot smooth curve
X_grid = np.linspace(-3, 3, 200).reshape(-1, 1)
y_grid_pred = model.predict(X_grid)
plt.scatter(X, y, label="Data")
plt.plot(X_grid, y_grid_pred, label="Polynomial fit")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows low MSE/high R^2 for a clear quadratic pattern and a curved fitted line.

Result / Conclusion Format

The Polynomial Regression model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
Overfitting with high degree	Use lower degree or cross-validation to choose degree.
Forgetting reshape	Use reshape(-1, 1) because sklearn expects 2D X.
Confusing polynomial with nonlinear algorithm	It is still linear regression on transformed features.

Practical Viva

Viva question	Correct answer / framing
Why use PolynomialFeatures?	It creates powers of features such as x^2 so linear regression can fit curves.
What happens if degree is too high?	The model may overfit training data and perform poorly on test data.
How to frame result?	Polynomial regression captured the curved relationship; MSE and R^2 show predictive performance.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 5: Lasso and Ridge Regression

Aim

To apply regularized regression methods and compare them with ordinary linear regression using MSE and R^2.

Theory

Ridge and Lasso reduce overfitting by adding penalties to coefficients. Ridge uses L2 penalty and shrinks coefficients. Lasso uses L1 penalty and can set some coefficients to zero, helping feature selection.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Diabetes dataset from sklearn with multiple numeric features.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Ridge model
ridge = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
ridge.fit(X_train, y_train)
ridge_pred = ridge.predict(X_test)

print("RIDGE REGRESSION")
print("MSE:", mean_squared_error(y_test, ridge_pred))
print("R2 Score:", r2_score(y_test, ridge_pred))
print("CV R2:", cross_val_score(ridge, X, y, cv=5, scoring="r2").mean())

# Lasso model
lasso = make_pipeline(StandardScaler(), Lasso(alpha=0.1, max_iter=10000))
lasso.fit(X_train, y_train)
lasso_pred = lasso.predict(X_test)

print("\nLASSO REGRESSION")
print("MSE:", mean_squared_error(y_test, lasso_pred))
print("R2 Score:", r2_score(y_test, lasso_pred))
print("CV R2:", cross_val_score(lasso, X, y, cv=5, scoring="r2").mean())
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output compares Ridge and Lasso using MSE, R^2, and mean cross-validation R^2.

Result / Conclusion Format

The Lasso and Ridge Regression model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
ConvergenceWarning in Lasso	Increase max_iter or adjust alpha.
Alpha too high	Very high alpha may underfit by shrinking too much.
No scaling	Regularized regression should usually be scaled so penalty is fair across features.

Practical Viva

Viva question	Correct answer / framing
What is regularization?	A penalty added to reduce overly large coefficients and overfitting.
Ridge vs Lasso?	Ridge uses L2; Lasso uses L1 and can make coefficients zero.
How to frame result?	Ridge/Lasso were trained to control overfitting; compare MSE/R^2 to judge performance.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 6: Logistic Regression

Aim

To build a logistic regression classifier and evaluate it with confusion matrix, accuracy, error, recall, specificity, F1-score, AUC, and cross-validation.

Theory

Logistic regression is used for classification. It estimates class probability using the sigmoid function. For binary classification, predicted probability is converted to class using a threshold.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Breast cancer dataset from sklearn. Target is malignant or benign class.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score
from sklearn.metrics import precision_score, f1_score, roc_auc_score

# Load data
data = load_breast_cancer()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create scaled logistic regression model
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)

# Predict class and probability
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Confusion matrix and metrics
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()
print("Confusion Matrix:\n", cm)
print("TP:", TP, "TN:", TN, "FP:", FP, "FN:", FN)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Error Rate:", (FP + FN) / (TP + TN + FP + FN))
print("Recall:", recall_score(y_test, y_pred))
print("Specificity:", TN / (TN + FP))
print("Precision:", precision_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))
print("AUC:", roc_auc_score(y_test, y_prob))

# Cross-validation accuracy
scores = cross_val_score(model, X, y, cv=5)
print("Mean CV accuracy:", scores.mean())
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows binary confusion matrix, TP/TN/FP/FN, accuracy, error rate, recall, specificity, precision, F1-score, AUC, and cross-validation accuracy.

Result / Conclusion Format

The Logistic Regression model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
LogisticRegression does not converge	Use StandardScaler and increase max_iter.
AUC error	AUC needs probabilities or decision scores, not only class labels.
Wrong TP/TN order	For binary confusion matrix in sklearn: [[TN, FP], [FN, TP]].

Practical Viva

Viva question	Correct answer / framing
Why logistic regression for classification?	It maps linear score to probability using sigmoid.
What is specificity?	TN / (TN + FP), ability to correctly identify negatives.
How to frame result?	The classifier predicts class probabilities and is evaluated using confusion matrix and AUC.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 7: Artificial Neural Network / MLP Classifier

Aim

To build a simple artificial neural network classifier using MLPClassifier and evaluate its performance.

Theory

An ANN contains connected neurons organized in layers. MLPClassifier is a feed-forward neural network with hidden layers. It learns nonlinear patterns using backpropagation and gradient-based optimization.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Digits dataset from sklearn. Each image is represented by pixel features and target is digit class 0-9.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Load digits data
digits = load_digits()
X = digits.data
y = digits.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create neural network model with scaling
model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(64,), activation="relu",
                  max_iter=500, random_state=42)
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# Cross-validation
scores = cross_val_score(model, X, y, cv=3)
print("Mean CV accuracy:", scores.mean())
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows multiclass confusion matrix, accuracy, classification report, and cross-validation accuracy. ANN usually performs well on digits but may take longer.

Result / Conclusion Format

The Artificial Neural Network / MLP Classifier model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
Training is slow	Reduce hidden layer size or use fewer CV folds.
ConvergenceWarning	Increase max_iter or scale data.
Poor accuracy	Check scaling, hidden_layer_sizes, max_iter, and random_state.

Practical Viva

Viva question	Correct answer / framing
What is hidden layer?	A layer between input and output that learns intermediate patterns.
What is backpropagation?	Method to compute gradients of error and update weights.
How to frame result?	The ANN learned nonlinear digit patterns from pixel features and was evaluated using accuracy and confusion matrix.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 8: K-NN Classifier

Aim

To classify data using K-Nearest Neighbors and evaluate it using classification metrics and cross-validation.

Theory

KNN predicts the class of a new sample by checking the k nearest training samples. It is distance-based, so feature scaling is important.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Iris dataset from sklearn.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Load data
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create KNN model with scaling
model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# Cross-validation
scores = cross_val_score(model, X, y, cv=5)
print("5-fold accuracy scores:", scores)
print("Mean CV accuracy:", scores.mean())
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows confusion matrix, accuracy, classification report, and cross-validation accuracy.

Result / Conclusion Format

The K-NN Classifier model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
Low accuracy due to k	Try different n_neighbors values such as 3, 5, 7.
No scaling	KNN is distance-based; use StandardScaler.
Slow prediction on large data	KNN stores training data and computes distances at prediction time.

Practical Viva

Viva question	Correct answer / framing
Why scale for KNN?	Because distance is affected by numeric range of features.
What does k mean?	Number of nearest neighbors used for voting.
How to frame result?	The model classified samples based on nearest training examples; accuracy and confusion matrix show performance.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 9: Decision Tree Classification

Aim

To train a decision tree classifier and evaluate it using classification metrics.

Theory

A decision tree splits data using feature conditions and predicts a class at leaf nodes. It is easy to interpret but can overfit if the tree is too deep.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Iris dataset from sklearn.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load data
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create and train decision tree
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Feature Importance:", model.feature_importances_)

# Cross-validation
scores = cross_val_score(model, X, y, cv=5)
print("Mean CV accuracy:", scores.mean())

# Plot tree
plt.figure(figsize=(12, 6))
plot_tree(model, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.show()
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows confusion matrix, accuracy, feature importance, cross-validation accuracy, and a plotted decision tree.

Result / Conclusion Format

The Decision Tree Classification model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
Tree overfits	Limit max_depth, min_samples_split, or use pruning.
plot_tree not visible	Use plt.show() and set a larger figure size.
Feature importance unclear	Each value indicates how much that feature helped split decisions.

Practical Viva

Viva question	Correct answer / framing
Why decision tree is interpretable?	It makes decisions through visible if-else-like feature splits.
What is pruning?	Reducing tree complexity to prevent overfitting.
How to frame result?	The tree learned decision rules for class labels and performance was measured using confusion matrix and accuracy.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 10: SVM Classification

Aim

To train a Support Vector Machine classifier and evaluate it using classification metrics and cross-validation.

Theory

SVM finds a hyperplane that separates classes with maximum margin. Kernel SVM can separate nonlinear data by mapping it into a higher-dimensional feature space.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Breast cancer dataset from sklearn for binary classification.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score
from sklearn.metrics import precision_score, f1_score, roc_auc_score

# Load data
data = load_breast_cancer()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Create SVM model with RBF kernel
model = make_pipeline(StandardScaler(), SVC(kernel="rbf", probability=True,
    random_state=42))
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Evaluate
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()
print("Confusion Matrix:\n", cm)
print("TP:", TP, "TN:", TN, "FP:", FP, "FN:", FN)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Error Rate:", (FP + FN) / (TP + TN + FP + FN))
print("Recall:", recall_score(y_test, y_pred))
print("Specificity:", TN / (TN + FP))
print("Precision:", precision_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))
print("AUC:", roc_auc_score(y_test, y_prob))

# Cross-validation
scores = cross_val_score(model, X, y, cv=5)
print("Mean CV accuracy:", scores.mean())
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows binary classification metrics and cross-validation accuracy. SVM often performs well after scaling.

Result / Conclusion Format

The SVM Classification model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
Slow training	SVM can be slow on large datasets; use smaller data or linear kernel.
AUC probability error	Set probability=True before fitting if using predict_proba.
Poor performance	Scale features and tune C/gamma/kernel.

Practical Viva

Viva question	Correct answer / framing
What is margin?	Distance between hyperplane and nearest support vectors.
What are support vectors?	Critical points closest to the boundary that determine the hyperplane.
How to frame result?	The SVM created a maximum-margin boundary and performance was reported using confusion matrix, F1, and AUC.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 11: K-means Clustering

Aim

To apply K-means clustering on unlabelled data and visualize clusters using two features or PCA components.

Theory

K-means is an unsupervised clustering algorithm. It groups data into k clusters by repeatedly assigning points to nearest centroids and updating centroids.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Iris dataset from sklearn. For clustering, target labels are not used for training. PCA is used for 2D visualization.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Load data
iris = load_iris()
X = iris.data

# Scale data because K-means uses distance
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Use PCA for 2D visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Apply K-means
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

print("Cluster labels:", clusters)
print("Inertia/WCSS:", kmeans.inertia_)
print("Cluster centers:\n", kmeans.cluster_centers_)

# Plot clusters
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-means Clustering on Iris")
plt.show()

# Elbow method
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(range(1, 11), inertias, marker="o")
plt.xlabel("Number of clusters k")
plt.ylabel("Inertia/WCSS")
plt.title("Elbow Method")
plt.show()
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows cluster labels, inertia, centers, a 2D cluster plot, and elbow method graph.

Result / Conclusion Format

The K-means Clustering model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
n_init warning	Set n_init=10 explicitly.
Clusters differ from class labels	Clustering is unsupervised; cluster numbers are arbitrary.
Bad cluster shape	K-means works best on compact spherical clusters and scaled data.

Practical Viva

Viva question	Correct answer / framing
Why not use y labels?	Clustering is unsupervised, so labels are not used for training.
What is inertia?	Sum of squared distances of samples to their nearest cluster center.
How to frame result?	K-means grouped samples into clusters based on similarity; elbow plot helps justify k.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Practical 12: Hierarchical Clustering

Aim

To perform hierarchical clustering and visualize the cluster hierarchy using a dendrogram.

Theory

Hierarchical clustering builds a tree of clusters. Agglomerative clustering starts with each point as its own cluster and merges the closest clusters step by step. A dendrogram visualizes these merges.

Required Libraries / Tools

- Python 3
- scikit-learn
- numpy
- pandas if CSV is used
- matplotlib for plots
- scipy for hierarchical dendrogram when required

Dataset Explanation

Iris dataset from sklearn. PCA is used for 2D plotting after clustering.

Step-by-Step Procedure

- Import required libraries.
- Load the dataset and separate input features X and target y, if target exists.
- For supervised learning, split data into training and testing sets.
- Scale features when the algorithm is distance-based or gradient-based.
- Create the model object with suitable parameters.
- Train the model using fit().
- Predict using predict() or predict_proba().
- Evaluate using the metrics required by the practical.
- Write a result/conclusion in simple words.

Complete Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Load data
iris = load_iris()
X = iris.data

# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Dendrogram using Ward linkage
linked = linkage(X_scaled, method="ward")
plt.figure(figsize=(10, 5))
dendrogram(linked)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.show()

# Agglomerative clustering
model = AgglomerativeClustering(n_clusters=3, linkage="ward")
clusters = model.fit_predict(X_scaled)
print("Cluster labels:", clusters)

# PCA for 2D visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Hierarchical Clustering on Iris")
plt.show()
```

Explanation of Important Lines

Code part	Meaning
load_*()	Loads a public built-in dataset from scikit-learn for reproducible practical work.
X and y	X contains input features; y contains target output for supervised learning.
train_test_split	Divides data into training and testing sets to evaluate generalization.
StandardScaler	Scales features so algorithms are not biased by large numeric ranges.
model = ...	Creates the machine learning algorithm object.
model.fit()	Trains the model using training data.
model.predict()	Generates predictions for unseen/test data.
metrics	Quantitatively judge model performance.
cross_val_score	Performs k-fold cross-validation to check stability of model performance.

Expected Output

Output shows dendrogram, cluster labels, and a 2D cluster visualization.

Result / Conclusion Format

The Hierarchical Clustering model was successfully implemented. The dataset was divided into training and testing parts, the model was trained, predictions were made, and performance was evaluated using suitable metrics. Based on the output, write whether the model performed well and mention the most important metric value.

Common Errors and Fixes

Error / issue	Fix
Dendrogram too crowded	Use smaller sample or truncate dendrogram for readability.
AgglomerativeClustering parameter error	In recent sklearn, use metric instead of affinity if needed; this code uses default with ward.
Cluster labels arbitrary	Cluster numbers do not mean actual class names.

Practical Viva

Viva question	Correct answer / framing
What is dendrogram?	A tree diagram showing how clusters merge at different distances.
What is agglomerative clustering?	Bottom-up hierarchical clustering that starts with individual points and merges them.
How to frame result?	The dendrogram helped visualize cluster hierarchy, and selected clusters were plotted after PCA.
What is the exam answer structure?	Aim -> theory -> dataset -> steps -> code -> output -> result -> viva explanation.
What should you say if score is low?	Mention possible reasons such as small dataset, unsuitable model, overfitting, lack of tuning, or noisy features.

Model Evaluation and Cross-Validation

Your practical guideline explicitly asks for training/test evaluation and k-cross-validation. These are extremely important for viva.

Train-Test Split

In train-test split, the dataset is divided into two parts. The model learns from the training set and is evaluated on the test set. This checks whether the model can generalize to unseen data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

- test_size=0.2 means 20% data is used for testing.
- random_state=42 makes the split reproducible.
- stratify=y preserves class ratio in classification datasets.

k-Fold Cross-Validation

In k-fold cross-validation, the dataset is divided into k parts/folds. The model is trained k times. Each time, one fold is used for testing and the remaining folds are used for training. The average score gives a more reliable estimate than a single split.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5)
print("Scores:", scores)
print("Mean score:", scores.mean())
```

Point	Train-test split	k-fold cross-validation
Speed	Faster	Slower because model trains k times
Reliability	Depends on one split	More reliable average score
Use	Quick practical evaluation	Better performance estimate

Final Revision Section

Quick Revision Checklist

Check	Can you answer this?
ML basics	Can you define feature, target, model, training, testing, generalization?
Learning types	Can you distinguish supervised, unsupervised, reinforcement learning?
Preprocessing	Can you explain scaling, feature selection, PCA, and data leakage?
Regression	Can you write equations for simple/multiple regression and explain MSE/R^2?
Regularization	Can you compare Ridge and Lasso?
Classification	Can you draw/explain TP, TN, FP, FN and metrics?
Algorithms	Can you explain working of NB, DT, KNN, SVM, Logistic, Perceptron, MLP?
Clustering	Can you explain K-means, hierarchical clustering, distance metrics, dendrogram?
Practicals	Can you write the common sklearn flow without looking?
Result writing	Can you conclude using metric values instead of vague statements?

Important Formulas

Formula / Must know

Simple Linear Regression: y_hat = b0 + b1*x
Multiple Linear Regression: y_hat = b0 + b1*x1 + ... + bp*xp
MSE = (1/n) * sum(yi - y_hat_i)^2
R^2 = 1 - SS_res / SS_total
Sigmoid: p = 1 / (1 + e^(-z))
Accuracy = (TP + TN) / total
Error rate = (FP + FN) / total
Recall = TP / (TP + FN)
Specificity = TN / (TN + FP)
Precision = TP / (TP + FP)
F1 = 2 * Precision * Recall / (Precision + Recall)
Euclidean distance = square root of sum of squared coordinate differences

Last-Minute Practical Exam Tips

- Always write X = features and y = target clearly.
- For classification, use stratify=y in train_test_split when possible.
- Scale data for KNN, SVM, logistic regression, neural networks, PCA, and clustering.
- Do not scale the full dataset before splitting; fit scaler on training data only for supervised learning.
- For regression, report MSE and R^2. Do not report accuracy for regression.
- For classification, report confusion matrix, accuracy, recall, specificity, F1-score, and AUC for binary classification.
- For clustering, remember there is no y during training. Cluster labels are not actual class labels.
- If output is not perfect, explain logically instead of panicking. Practical exams test understanding too.
- Write result using metric values: e.g., 'The model achieved 0.95 accuracy, so it classified most test samples correctly.'

- Keep imports, model creation, fit, predict, metrics in the same clean order.

Most Expected Viva Questions

Viva question	Correct answer / framing
What is the difference between supervised and unsupervised learning?	Supervised uses labelled target y; unsupervised finds patterns without y.
What is train-test split?	Dividing data into training part for learning and test part for evaluation.
Why use cross-validation?	To get a more reliable performance estimate across multiple splits.
What is overfitting?	Model memorizes training data and performs poorly on unseen data.
How to prevent overfitting?	Regularization, cross-validation, pruning, simpler model, more data, feature selection.
Why scaling?	It puts features on comparable scales and helps distance/gradient algorithms.
What is PCA?	Dimensionality reduction technique creating principal components that capture maximum variance.
What metrics for regression?	MSE and R^2 score.
What metrics for classification?	Confusion matrix, accuracy, error, recall, specificity, precision, F1, AUC.
What is Naive Bayes?	Probability-based classifier using Bayes theorem and independence assumption.
Why logistic regression is classification?	Because sigmoid converts score to probability and threshold gives class.
What is KNN?	Classifies based on majority class among k nearest neighbors.
What is SVM?	Maximum-margin classifier that finds best separating hyperplane.
What is ANN?	Network of connected neurons/layers trained to learn complex patterns.
What is K-means?	Unsupervised algorithm that partitions data into k clusters using centroids.
What is hierarchical clustering?	Builds a tree of clusters and visualizes it using dendrogram.

Final Model Answer Template

```
Aim:
To implement <algorithm name> on <dataset name> and evaluate its performance.

Theory:
<Algorithm name> is used for <regression/classification/clustering>. It works by...

Procedure:
1. Import libraries.
2. Load dataset.
3. Separate X and y if supervised.
4. Split into train and test sets if supervised.
5. Apply preprocessing such as scaling if needed.
6. Create model and train using fit().
7. Predict using predict().
8. Evaluate using suitable metrics.

Result:
The model was successfully implemented. The obtained <metric> was <value>.
This means <simple interpretation>.
```

End of guide.